

Задание «Клиент-серверное приложение на основе WebSocket. Бизнес-протокол на основе Protobuf.»

- Загружать решения в виде ZIP архива в проект Smart LMS под названием HW5
- В архиве должна быть папка, которая включает в себя проект, отчет¹ (DOCX/PDF формат) и файл со ссылкой на видео (см. ниже)
- В одном проекте иметь и код клиента и код сервера
- Название ZIP архива должно совпадать с Вашей фамилией.

Мотивация.

Одна из наиболее частых задач программной инженерии при использовании компьютерных сетей – создание

- (1) собственного протокола (бизнес-протокола, custom protocol) вместе с
- (2) собственным программным клиентом и сервером (работающими по новому протоколу с поддержкой множества клиентов, которые разработаны на языках и работают платформах, отличных от языка и платформы сервера)

Протоколы на основе JSON либо подобных форматах являются гибкими, однако имеют ряд недостатков, о которых мы говорили на лекции о Protocol Buffers.

В этом задании Вам следует познакомиться с:

- (1) best-practices создания сетевых протоколов на Application Layer (OSI)
- (2) межязыковым сетевым взаимодействием на примере JavaScript $\leftrightarrow$ выбранный язык
- (3) Netty либо иной Framework. При разработке клиент-серверного приложения Вы задействуете Netty либо иной Framework себе в помощь и продолжаете знакомство с лучшими архитектурными patterns разработки сетевых программных серверов.

Мы глубоко рассмотрели протокол WebSocket и приемы межязыковой, межплатформенной сетевой коммуникации на примере Protocol Buffers. Также на лекциях и семинарах рассматривался Netty Framework, но многие сетевые шаблоны проектирования (networking design patterns) – общие для разных сетевых программных Frameworks. Понимание Netty Framework важно, поскольку затрагивает и проясняет много сетевых особенностей на достаточно глубоком уровне для программных инженеров.

Netty Framework – один из наиболее популярных Framework для разработки сетевых приложений. Его используют многие ведущие компании, включая RedHat, Twitter, Infinispan, HornetQ, Vert.x, Finagle, Akka, Yandex, Apache Cassandra, Elasticsearch, Facebook. Проект активно развивается уже долгие годы, почти каждый день есть обновления (commits): <https://github.com/netty/netty>

¹ Примечание: без заполнения ключевых разделов отчета, напр. об использовании ИИ, работа может быть оценена с коэффициентом 0.5 (т. е., по умолчанию будет считаться, что не были выполнены ключевые требования задания либо был использован ИИ, шаблон отчета находится в конце этого файла)

Язык программирования и сетевой Framework для серверной части.

В работе можно использовать один из следующих вариантов:

- Java / Netty Framework (оценка до 10 баллов)
- Rust / Actix-Web / Actix-Net (оценка до 10 баллов если без await)
- C++ / Boost.Asio (оценка до 10 баллов если без coroutines)
- Kotlin / Ktor (не рекомендуются, оценка до 9 баллов²)
- Go / net.Conn / bufio (не рекомендуются, оценка до 9 баллов²)

Python использовать запрещено: при разработке программного приложения слишком много важных принципов сетевой коммуникации скрыто от разработчика.

Язык программирования для клиентской части – чистый JavaScript (без Frameworks)**Оценивание задания:**

- код будет анализироваться на корректность и точность выполнения требований этого задания
- будет выполнен анализ полученных результатов, отчета и видео
- будет учитываться качество кода (форматирование, обработка corner cases, иные стандартные элементы качества кода)

² Kotlin использует coroutines, Go использует goroutines: хотя на практике код выглядит проще, в образовательных целях это препятствует углубленному пониманию асинхронности потоков данных в компьютерных сетях и возникающих при этом проблем.

Содержание задания.

Общие требования:

- Клиент – чат на чистом JavaScript (без frameworks)
- Сервер – см. выше
- Протокол общения – WebSocket (Binary Mode)
- Пользователь чата может вводить сообщения и отправлять их серверу, возможно также по желанию прикреплять к сообщению изображение. Сервер дублирует сообщение с изображением (при его наличии) всем остальным пользователям чата.
- Изображения – файлы JPEG/PNG размером до 1 МБ.

ЗАДАНИЕ НА ОЦЕНКУ ДО 7 ЛИБО 8 БАЛЛОВ (В ЗАВИСИМОСТИ ОТ ВЫБРАННОГО ЯЗЫКА И FRAMEWORK)

Требования к клиенту:

- Единая HTML страница вместе с кодом JavaScript
- В одном браузере можно открыть несколько HTML страниц (это будут разные клиенты чата)
- HTML страница содержит 3 элемента:
 - Тестовая область (1 строка) для ввода сообщений (до 25 символов)
 - Тестовая область (10 строк, readonly, по желанию можно добавить прокрутку) для просмотра сообщений от всех пользователей
 - Графическая область для вывода изображений, которые могут быть отправлены вместе с текстом (можно соединить текстовую и графическую область в один логический элемент GUI)
 - Кнопка «отправить», которая отправляет всем пользователям чата текстовое сообщение из строки ввода (сообщение отображается у всех пользователей включая отправителя в текстовой области для просмотра сообщений; добавляется к уже имеющимся сообщениям)
 - Кнопка «прикрепить», которая позволяет по желанию к текстовому сообщению прикрепить изображение
- Сразу при открытии HTML страницы клиент пытается установить соединение с сервером по протоколу WebSocket
- При успешном подключении выводится история сообщений, полученная от сервера
- Если соединение установить не удалось, то
 - через 10 секунд повторить попытку установки соединения (использовать таймер)
 - строка ввода, кнопки «отправить» и «прикрепить» - неактивны, выведено на странице соответствующее сообщение и оставшееся число секунд (автоматически обновляемое поле) до повтора попытки соединения

Требования к серверу:

- Использовать выбранный выше язык, Framework, протокол WebSocket
- Поддерживать соединения от нескольких пользователей (до 100)
- Реализовать логику приема и дублирования сообщений от пользователей
- Вести историю сообщений (до 50 строк и 50 изображений)
- При подключении нового пользователя отправлять ему историю сообщений и последнее изображение (при наличии)

Требования к бизнес-протоколу:

- Необходимо разработать свой собственный бизнес-протокол (custom, business protocol) на основе Protocol Buffers.
- Клиент-серверное взаимодействие осуществляется с помощью Protocol Buffers. Требуется использовать WebSocket BINARY MODE.
- Каждое сообщение серверу – запрос, каждое сообщение от сервера – ответ. Формат сообщений Protocol Buffers разработайте самостоятельно – это часть задания, пример – см. проекты на GitHub (ссылки ниже, мы рассматривали протокол по ссылкам на лекции).

Продолжение требований к бизнес-протоколу на основе Protocol Buffers:

- использовать синтаксис **proto3**
- иметь в бизнес протоколе несколько сообщений (message)
- иметь в бизнес протоколе вложенные сообщения
- хотя бы в одном сообщении должны быть поля со значением default
- должны быть поля repeated (напр., для изображений)
- должна быть обработка на стороне сервера всех optional сообщений (т.е., проверка в коде сервера присутствия в данном сообщении вложенного сообщения либо optional поля в сообщении)
- должны быть использованы несколько типов полей (напр., int32, bool)

Задание на оценку до 9 либо 10 баллов (в зависимости от выбранного языка и Framework)

Дополнительные требования к клиенту и серверу:

- Появляются дополнительные: поле «имя» («name») и кнопка «присоединиться»
- Немного изменяется сценарий подключения пользователя к чату: не сразу при открытии страницы, а после ввода имени и нажатии кнопки «присоединиться»
- Алгоритм подключения к серверу аналогичен предыдущему заданию
- После успешного подключения клиент отправляет серверу запрос – «присоединиться к чату» с указанием своего имени
- Если в чате нет пользователя с таким именем, сервер отправляет пользователю ответ – история сообщений. Иначе, приходит ответ – «ошибка» с указанием причины ошибки.
- Далее обычные сообщения посылаются клиентами в виде запросов и приходят другим клиентам в виде сообщений от сервера.
- Пользователь может написать @<name> <текст сообщения>. Тогда формируется соответствующий запрос серверу, чтобы введенный <текст сообщения> отправлялся только пользователю с именем <name>.
- Сервер возвращает ошибку (ответ с соответствующим кодом), если в чате сейчас нет пользователя с именем <name>, которому адресовано сообщение (клиент должен отобразить ошибку соответствующим образом).
- Вся коммуникация по-прежнему происходит по Protocol Buffers и WebSocket (binary mode)
- К каждому сообщению можно прикрепить изображение. Если изображение прикрепляется при подключении, то оно служит icon пользователя (выводить это изображение другим пользователям в чате при получении сообщения от соответствующего пользователя). Можно дополнительно ограничить формат и размер изображений для icon. Пользователь не обязан при подключении прикреплять изображение, но такая возможность должна быть.

Примеры JavaScript Client & Netty Java Server (работают на WebSocket и используют Protocol Buffers):

<https://github.com/Wikience/WRRS-JS>

<https://github.com/Wikience/RServer>

Работа с JSON, WebSocket – лекция и учебник:

<https://learn.javascript.ru/websockets>

<https://learn.javascript.ru/json>

Почему WebSocket?

It enables fast, asynchronous, real-time Web I/O
Its support is ubiquitous



<https://caniuse.com/#feat=websockets>

Chrome	Edge [*]	Safari	Firefox	Opera	IE
		3.1-4	2-3.6		
¹ 4-14		¹ 5-5.1	¹ 4-5	10.1	
² 15		² 6-6.1	² 6-10	¹ 11.5	6-9
16-111	12-111	7-16.3	11-111	12.1-94	10
112	112	16.4	112	95	11
113-115		16.5-TP	113-114		

Шаблон отчета

ОТЧЕТ ПО HW5

ФИО, группа, email

Цель работы: *укажите цель работы своими словами*

Я претендую на оценку до 5 баллов, был использован генеративный ИИ (далее следует информация из декларации согласно https://www.hse.ru/studyspravka/ai_guidelines/)

ЛИБО

Я претендую на оценку до 9 (10) баллов (зависит от выбранных языка программирования и сетевого Framework), генеративный ИИ не был использован для работы над этим заданием.

Укажите в тексте отчета выбранные язык программирования и сетевой Framework, а также на какую оценку Вы претендуете (до 7, 8, 9, 10).

Дополнительно требуется записать видео для демонстрации работы приложения (по всем требованиям задания). Видео необходимо разместить на RuTube/VKVideo (видео на иностранных платформах не принимаются). Ссылка на видео должна находиться в файле с именем <фамилия_видео.doc> (формат DOC). Файл должен находиться в корне папки с проектом.

На видео должна быть показана конфигурация программного кода, запуск приложения, детальная демонстрация работоспособности и клиента и сервера.

Цель видео – доказать, что Ваше приложение корректно функционирует согласно всем требованиям.

Кратко опишите архитектуру решения и разработанный бизнес-протокол. Обоснуйте принятые решения.

Приведите ключевые фрагменты кода (основные классы) реализации бизнес протокола и коммуникации между клиентом и сервером.

Видео должно сопровождаться Вашими подробными голосовыми комментариями о том, что происходит и что демонстрируется на видео.

Длительность видео – от 15 до 30 минут.